CSE385: Analysis of Algorithms (Honors), Spring 2026 **Date:** March 23, 2026

Midterm Exam (Solution Sketch)

(6:35 PM – 7:50 PM : 75 Minutes)

- This exam will account for 25% of your overall grade.
- There are five (5) questions, worth 75 points in total. Please answer all of them in the spaces provided.
- There are 24 pages including 13 blank pages. Please use the blank pages if you need additional space for your answers.
- This is a *closed book*, *closed notes* exam. *No cheat sheets* are allowed.
- You are *not allowed to use calculators*.
- You may use, without proof, any results that were covered in class or included in the homework solutions provided by the instructor.

Good Luck!

Question	Pages	Parts	Points	Score
1. Recurrences	2 – 5	(a) – (c)	$3 \times 4 = 12$	
2. Asymptotic Bounds	6 – 9	(a) – (d)	$4 \times 3 = 12$	
3. A Simple Iterative Function	10 – 15	(a) – (c)	$5 + 5 + 10 = 20$	
4. Quadsort	16 – 19	(a) – (b)	$7 + 9 = 16$	
5. Improving Karatsuba's Algorithm	20 – 24	(a) – (c)	$4 + 8 + 3 = 15$	
Total	–	–	75	

Name: _____

SBU ID: _____

Question 1. [**12 Points**] **Recurrences.** For each of the functions below find $T(n)$ using the Master Theorem. Assume that $n = 2^k$ for some integer $k \geq 0$.

(a) [**4 Points**] $T(n) = \begin{cases} \Theta(1) & \text{if } n < 8, \\ T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + 2\,T\left(\frac{n}{8}\right) + n & \text{otherwise.} \end{cases}$

(b) [**4 Points**] $T(4^n) = \begin{cases} \Theta(1) & \text{if } n < 2, \\ 2\,T(2^n) + n & \text{otherwise.} \end{cases}$

(c) [**4 Points**] $T(n) = \begin{cases} \Theta(1) & \text{if } n < 4, \\ 3\,T\left(\frac{n}{2}\right) + n & \text{if } n \geq 4 \text{ and } \log_2 n \text{ is even,} \\ 2\,T\left(\frac{n}{2}\right) + n \log n & \text{otherwise.} \end{cases}$

(a) Define $S(n) = T(n) + T(\frac{n}{2}) + T(\frac{n}{4})$. Then,

$$\begin{aligned} S(n) &= T(n) + T(\frac{n}{2}) + T(\frac{n}{4}) \\ &= 2T(\frac{n}{2}) + 2T(\frac{n}{4}) + 2T(\frac{n}{8}) + n \\ &= 2S(\frac{n}{2}) + n. \end{aligned}$$

By the Master theorem, we have $S(n) = \Theta(n \log n)$.

Now, $2T(n) = T(n) + T(\frac{n}{2}) + T\left(\frac{n}{4}\right) + 2\,T\left(\frac{n}{8}\right) + n = S(n) + 2T(\frac{n}{8}) + n \geq S(n)$. Therefore, $T(n) \leq S(n) \leq 2T(n)$. Therefore, $T(n) = \Theta(S(n)) = \Theta(n \log n)$.

(b) Define $S(n) = T(4^n)$. Then, $T(n) = S(\log_4 n)$ and $T(2^n) = S(\frac{n}{2})$. Thus,

$$\begin{aligned} S(n) &= T(4^n) \\ &= 2T(2^n) + n \\ &= 2S(\frac{n}{2}) + n. \end{aligned}$$

By the Master theorem, we have $S(n) = \theta(n \log n)$, and $T(n) = S(\log_4 n) = \theta(\log n \log\log n)$.

(c) First consider the case: $\log_2 n$ is even. Then, $\log_2 \frac{n}{2} = \log_2(n) - 1$ is odd. Therefore,

$$\begin{aligned} T(n) &= 3T(\frac{n}{2}) + n && [\log_2 n \text{ is even}] \\ &= 3\left(2T(\frac{n}{4}) + \frac{n}{2}\log\frac{n}{2}\right) + n && [\log_2 \frac{n}{2} \text{ is odd}] \\ &= 6T(\frac{n}{4}) + \theta(n \log n). \end{aligned}$$

Similarly, we can show that when $\log_2 n$ is odd, the same recurrence holds, and so it holds for all $n \geq 4$. By the Master theorem, we have $T(n) = \theta(n^{\log_4 6})$.

Question 2. [**12 Points**] **Asymptotic Bounds.** For each of the following statements, state whether it is *True* or *False*. Justify each answer. Assume that all log's are of base 2.

(*a*) [**3 Points**] $\sum_{i=1}^{n} \frac{1}{i^2} = \omega\left(\sum_{i=1}^{n} \frac{1}{i^3}\right)$

(*b*) [**3 Points**] $2^{(\sin n)^2 \log n} = \Theta\left(\frac{n}{2^{(\cos n)^2 \log n}}\right)$

(*c*) [**3 Points**] $n^{\frac{\log\log n}{\log n}} = \Theta\left((\log n)^{1+\frac{1}{\log\log n}}\right)$

(*d*) [**3 Points**] $\Theta(n) - \Theta(n) = o(n)$

(a) **False**. From homework remember that, $\sum_{i=1}^{n} \frac{1}{i^2} = \theta(1)$. Since $\frac{1}{i^3} \leq \frac{1}{i^2} \quad \forall i \geq 1$, we must have $\sum_{i=1}^{n} \frac{1}{i^3} \leq \sum_{i=1}^{n} \frac{1}{i^2} = \theta(1)$. Thus both sums are $\theta(1)$.

(b) **True**. $2^{(\sin n)^2 \log n} = (2^{\log n})^{sin^2 n} = n^{sin^2 n} = n^{1-cos^2 n} = \frac{n}{n^{cos^2 n}} = \frac{n}{2^{cos^2 n \log n}}$.

(c) **True**. $n^{\frac{\log\log n}{\log n}} = 2^{\log n \frac{\log\log n}{\log n}} = 2^{\log\log n} = \log n = \theta(\log n)$.
On the other hand, $(\log n)^{1+\frac{1}{\log\log n}} = 2^{\log\log n(1+\frac{1}{\log\log n})} = 2^{\log\log n+1} = 2\log n = \theta(\log n)$

(d) **False**. $3n = \theta(n)$, and $2n = \theta(n)$, but $3n - 2n = n$ which is not $o(n)$.

```
LOOP-F( A[0 : n] )
Input: An array A[0 : n] of length n + 1
Output: Updated array A[0 : n]
1. for t ← 1 to n do
2.     for r ← n downto 1 do
3.         A[r] ← A[r − 1] + A[r]
```

Figure 1: [Question 3] A simple iterative function.

QUESTION 3. [20 Points] A Simple Iterative Function. Figure 1 shows a simple iterative function LOOP-F that updates an array $A[0:n]$ of length $n+1$.

(a) **[5 Points]** What is the running time of LOOP-F($A[0:n]$)?

Both loops execute n times, thus line 3 runs n^2 times. So the runtime is $\theta(n^2)$.

(b) **[5 Points]** Suppose before we call LOOP-F($A[0:n]$), we set $A[0] = 1$ and $A[i] = 0$ for $1 \leq i \leq n$. Show that $A[r] = \binom{n}{r}$ for each $r \in [0, n]$ after LOOP-F finishes execution.

Hint. *For non-negative integers n and r and $1 \leq r \leq n$, $\binom{n}{r} = \binom{n-1}{r} + \binom{n-1}{r-1}$, where $\binom{n}{r} = \frac{n!}{r!(n-r)!}$ is the number of ways one can choose r objects from n objects.*

Solution outline: The key observation is that after the ith iteration of the outer loop finishes, A contains the i th row of the Pascal's triangle. In other words, after i iterations of the outer loop, $A[r] = \binom{i}{r}$. This can be proven by induction, using the fact that $\binom{i}{r} = \binom{i-1}{r} + \binom{i-1}{r-1}$.

(*c*) [**10 Points**] Suppose that before we call LOOP-F($A[0:n]$), we set each $A[i]$ to an arbitrary nonnegative value, where $0 \leq i \leq n$. Explain how LOOP-F can be modified to run in $\Theta(n \log n)$ time and generate exactly the same output that the original code would have generated. No need to write pseudocode.

Hint. *Frame each iteration of the outermost loop as a polynomial multiplication problem. For n iterations the product will look like $S(\ldots(S(S(SA)))\ldots) = S^n A$, where both S and A are polynomials. Also, note that $(1+x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k$ for any non-negative integer n.*

As stated in the hint, we need to compute $S^n A = (1+x)^n A$. There are $n+1$ polynomials, so we need n polynomial multiplications, if we multiply them one by one. If we use FFT to multiply polynomials in $\theta(n \log n)$, this takes $\theta(n^2 \log n)$ time.

To improve the runtime, we will calculate $(1+x)^n$ faster. There are a few ways to do so.

- We can use divide and conquer. We can calculate $(1+x)^n$ by recursively calculating $(1+x)^{\lfloor \frac{n}{2} \rfloor}$, and then multiplying to itself using FFT. This takes $\theta(n \log n)$ time.
- Alternatively, we can find the point value form of $(1+x)$ using FFT, treating it as a degree n polynomial. Then, we can find the point value form of $(1+x)^n$ simply by exponentiating each value. Finally, we can find the coefficients of $(1+x)^n$ doing an inverse FFT. This whole process takes $\theta(n \log n)$ time.
- Finally, we can use the hint that, $(1+x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k$. We need to calculate $\binom{n}{k}$ for each $k = 0 \cdots n$ quickly. We use the recurrence, $\binom{n}{k} = \frac{n-k+1}{k} \binom{n}{k-1}$ allowing us to find all $\binom{n}{k}$ in $O(n)$.

Once we have found $(1+x)^n$ efficiently, we can multiply it with A using FFT in $\theta(n \log n)$.

Question 4. [16 Points] QuadSort. The QuadSort algorithm shown below can be used to sort an array of numbers $A[1:n]$, where $n = 4^k$ for some integer $k \geq 0$.

QuadSort($A,\ p,\ r$)

Input: An array $A[p:r]$ of $n = r - p + 1$ numbers, where $n = 4^k$ for some integer $k \geq 0$.

Output: The numbers in $A[p:r]$ sorted in nondecreasing order of value.

Algorithm:

```
1.  n ← r − p + 1
2.  if n > 1 then
3.      m ← n/4
4.      QuadSort( A, p, p + m − 1 )          {recursively sort the 1st quarter of A[p : r]}
5.      QuadSort( A, p + m, p + 2m − 1 )     {recursively sort the 2nd quarter of A[p : r]}
6.      QuadSort( A, p + 2m, p + 3m − 1 )    {recursively sort the 3rd quarter of A[p : r]}
7.      QuadSort( A, p + 3m, p + 4m − 1 )    {recursively sort the 4th quarter of A[p : r]}
8.      Merge( A, p, p + m, m )              {merge the 1st and 2nd quarters of A[p : r]}
9.      Merge( A, p + 2m, p + 3m, m )        {merge the 3rd and 4th quarters of A[p : r]}
10.     Merge( A, p, p + 2m, 2m )            {merge the 1st and 2nd halves of A[p : r]}
```

Merge($A,\ p_1,\ p_2,\ n$)

Input: Two disjoint subarrays $A[p_1 : p_1 + n - 1]$ and $A[p_2 : p_2 + n - 1]$ each with n numbers sorted in nondecreasing order of value, where $n = 2^k$ for some integer $k \geq 0$.

Output: The smallest n of the $2n$ numbers in sorted order (nondecreasing order of value) in $A[p_1 : p_1 + n - 1]$ and the largest n numbers in sorted order in $A[p_2 : p_2 + n - 1]$.

Algorithm:

```
1.  if n ≥ 1 then
2.      if n = 1 then
3.          if A[p1] > A[p2] then A[p1] ↔ A[p2]      {swap A[p1] and A[p2]}
4.      else
5.          m ← n/2
6.          Merge( A, p1, p2, m )                   {merge the left halves of the subarrays}
7.          Merge( A, p1 + m, p2 + m, m )           {merge the right halves of the subarrays}
8.          Merge( A, p1 + m, p2, m )               {merge right half of A[p1 : p1 + n − 1]
                                                     with left half of A[p2 : p2 + n − 1]}
```

(a) **[7 Points]** Let $M(n)$ be the worst-case running time of Merge for merging two subarray of length n each. Write a recurrence relation for $M(n)$ and solve it.

$$M(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 3M\left(\frac{n}{2}\right) + \mathcal{O}(1) & \text{otherwise.} \end{cases}$$

Using the Master Theorem, $M(n) = \Theta\left(n^{\log_2 3}\right)$.

(*b*) [**9 Points**] Let $Q(n)$ be the worst-case running time of QUADSORT for sorting an array of length n. Write a recurrence relation for $Q(n)$ and solve it.

$$Q(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 4Q\left(\frac{n}{4}\right) + 2M(n/4) + M(n/2) = 4Q\left(\frac{n}{4}\right) + \Theta\left(n^{\log_2 3}\right) & \text{otherwise.} \end{cases}$$

Using the Master Theorem, $T(n) = \Theta\left(n^{\log_2 3}\right)$.

Question 5. [15 Points] Improving Karatsuba's Algorithm. We will modify Karatsuba's algorithm for multiplying two n-bit integers by computing and saving some small products ahead of time. We will use this version for $n \geq m$ only, where $m = 2^l$ for some fixed integer $l \geq 0$.

Precomputation (only once before the first use of Modified-Karatsuba **below).** Allocate a 2D array S, and for every possible pair $\langle x, y \rangle$ of m-bit integers, compute xy in $\Theta\left(m^2\right)$ time and set $S[x, y] \leftarrow xy$.

Modified-Karatsuba(x, y, n)

Input: n-bit integers x and y, where n is a power of 2 and $n \geq m$

Output: xy

1. **if** $n = m$ **then return** $S[x, y]$
2. **else** multiply x and y recursively as in standard Karatsuba and **return** the product

Figure 2: [Question 5] Modified Karatsuba.

(*a*) [**4 Points**] Analyze the precomputation time and space (i.e., size of S in bits) of the algorithm.

Space: The 2D array S stores the product for every pair of m-bit integers.

- The number of distinct m-bit integers is 2^m.
- The number of pairs of m-bit integers is $2^m \times 2^m = 2^{2m}$.
- The product of two m-bit integers is at most a $2m$-bit integer.
- Total space = (number of entries) $\times$ (bits per entry) $= 2^{2m} \times 2m = \Theta\left(m \cdot 2^{2m}\right)$ bits.

Time: We must compute the product for each of the 2^{2m} pairs. The cost depends on the multiplication algorithm used for this precomputation.

- Number of multiplications to perform is 2^{2m}.
- The problem does not specify the algorithm for the precomputation.
  - Using the standard Karatsuba's algorithm itself, the time would be $\Theta\left(m^{\log_2 3}\right)$.
  - Using regular multiplication, the time would be $\Theta\left(m^2\right)$.
- The total time is therefore either:
  - $2^{2m} \times \Theta\left(m^{\log_2 3}\right) = \Theta\left(m^{\log_2 3} \cdot 2^{2m}\right)$ or
  - $\Theta\left(m^2 \cdot 2^{2m}\right)$

(*b*) [**8 Points**] Let $T(n)$ be the time needed by MODIFIED-KARATSUBA to multiply two n-bit integers. Write a recurrence relation for $T(n)$ and solve it to show that $T(n) = \Theta\left(\left(\frac{n}{m}\right)^{\log_2 3}\right)$.

Recurrence Relation: The algorithm follows standard Karatsuba for $n > m$, making 3 recursive calls on inputs of size $n/2$ and doing $\Theta(n)$ work for additions. The recursion stops at the base case $n = m$, where it performs a table lookup in $\Theta(1)$ time. The recurrence is:

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = m, \\ 3\,T(n/2) + \Theta(n), & \text{if } n > m. \end{cases}$$

Solving the recurrence: We can solve this by analyzing the recursion tree/other methods. Here's a solution with recursion tree. Let the recursion have a depth of k where the problem size is reduced from n to m. Then,

$$\frac{n}{2^k} = m \implies 2^k = \frac{n}{m} \implies k = \log_2\left(\frac{n}{m}\right)$$

The number of leaves in the recursion tree is 3^k. The cost at each leaf is $T(m) = \Theta(1)$.

$$\begin{aligned} \text{Total work at the leaves} &= \#\text{leaves} \times \text{work per leaf} \\ &= 3^k \times \Theta(1) \\ &= \Theta\left(3^{\log_2(n/m)}\right) \\ &= \Theta\left(\left(2^{\log_2 3}\right)^{\log_2(n/m)}\right) \\ &= \Theta\left(\left(2^{\log_2(n/m)}\right)^{\log_2 3}\right) \quad (\text{since } (x^a)^b = (x^b)^a) \\ &= \Theta\left(\left(\frac{n}{m}\right)^{\log_2 3}\right) \end{aligned}$$

$$
\begin{aligned}
\text{Total work at internal nodes} &= \sum_{i=0}^{k-1} \#\text{internal nodes at level } i \times \text{work per internal nodes at level } i \\
&= \sum_{i=0}^{k-1} 3^i \times \Theta\left(\frac{n}{2^i}\right) \\
&= \Theta\left(n \sum_{i=0}^{k-1} \left(\frac{3}{2}\right)^i\right) \\
&= \Theta\left(n \times \frac{\left(\frac{3}{2}\right)^k - 1}{\frac{3}{2} - 1}\right) \\
&= \Theta\left(n \times \left(\frac{3}{2}\right)^k\right) \\
&= \Theta\left(n \times \frac{3^k}{2^k}\right) \\
&= \Theta\left(n \times \frac{\left(\frac{n}{m}\right)^{\log_2 3}}{\frac{n}{m}}\right) \\
&= \Theta\left(m \times \left(\frac{n}{m}\right)^{\log_2 3}\right)
\end{aligned}
$$

$$
\begin{aligned}
\text{Total work} &= \text{work done at leaves} + \text{work done at internal nodes} \\
&= \Theta\left(\left(\frac{n}{m}\right)^{\log_2 3}\right) + \Theta\left(m \times \left(\frac{n}{m}\right)^{\log_2 3}\right) \\
&= \Theta\left(m \times \left(\frac{n}{m}\right)^{\log_2 3}\right) \\
&= \Theta\left(\frac{n^{\log_2 3}}{m^{\log_2 (3/2)}}\right)
\end{aligned}
$$

Thus, $T(n) = \Theta\left(\frac{n^{\log_2 3}}{m^{\log_2 (3/2)}}\right)$.

(*c*) [**3 Points**] If you are allowed to allocate only $n \log_2 n$ bits of space for S, what is the best bound you can achieve for $T(n)$?

Hint. *What is the largest value of m implied by the given size of S?*

From part (a), the space required for the table S is $\Theta\left(m \cdot 2^{2m}\right)$. We are given a space constraint of $n \log_2 n$. To find the largest possible m, we set them equal:

$$m \cdot 2^{2m} \approx n \log_2 n$$

The exponential term 2^{2m} dominates the left side, so we can approximate:

$$2^{2m} \approx n \log_2 n$$

Taking $\log_2$ of both sides:

$$\log_2(2^{2m}) \approx \log_2(n \log_2 n) 2m \approx \log_2 n + \log_2(\log_2 n)$$

Asymptotically, the $\log_2 n$ term is dominant. Thus, $2m = \Theta(\log_2 n)$, which implies $m = \Theta(\log_2 n)$. Since the precomputation step uses precisely $2m \times 2^{2m}$ bits of space, setting $2m \times 2^{2m} = n \log_2 n$ gives us precisely $m = \frac{1}{2} \log_2 n$. But since m must be an integer, we should settle for $m = \left\lfloor \frac{1}{2} \log_2 n \right\rfloor$.
Now, we substitute this value of m into the time complexity from part (b):

$$T(n) = \Theta\left(\frac{n^{\log_2 3}}{m^{\log_2 (3/2)}}\right) = \Theta\left(\frac{n^{\log_2 3}}{(\log_2 n)^{\log_2 (3/2)}}\right)$$

This is the best time-bound achievable with the given space constraint.